Bilgisayar Programlama

Temel ve İleri Düzey Operatörler

İçindekiler (Bölüm 1: Operatörler)

1. Temel Operatörler
   - Aritmetik Operatörler
   - İlişkisel (Karşılaştırma) Operatörler
   - Mantıksal Operatörler
   - Arttırma ve Azaltma Operatörleri
   - Atama Operatörleri
2. İleri Düzey Operatörler
   - Bit Düzeyi (Bitwise) Operatörler
   - `sizeof` ve Virgül (`,`) Operatörleri
3. Operatör Önceliği ve Tip Dönüşümleri
   - Öncelik (Precedence) ve Birleşme (Associativity)
   - Otomatik ve Bilinçli Tip Dönüşümleri (Casting)

BÖLÜM 1: OPERATÖRLER

1.1. Operatör Kavramına Giriş

Operatörler, derleyiciye belirli bir matematiksel veya mantıksal işlemin gerçekleştirilmesini bildiren sembollerdir . C dili, 45 farklı operatöre sahip zengin bir operatör seti sunar . Operatörler, işlenen (operand) sayısına göre üç ana kategoriye ayrılır:

- **Unary (Tek İşlenenli) Operatörler**: Tek bir değişken veya sabit üzerinde işlem yapar (örnek: `-a` , `++x`)
- **Binary (İki İşlenenli) Operatörler**: İki değer arasında işlem yapar (örnek: `a + b` , `x > y`)
- **Ternary (Üç İşlenenli) Operatör**: Tek bir operatör olan `? :` (koşul operatörü) bu gruba girer

1.2. Aritmetik Operatörler (Arithmetic Operators)

Matematiksel hesaplamaların temel yapı taşlarıdır. C dilinde bu operatörler iki ana kategoriye ayrılır: **Unary** (Tekli) ve **Binary** (İkili).

1.2.1. Temel Operatör Tablosu

Operatör	Tür	İşlev	Örnek	Açıklama
`+`	Unary	Unary Plus	`+a`	Sayının işaretini korur (opsiyonel)
`-`	Unary	Unary Minus	`-a`	Sayının işaretini tersine çevirir
`+`	Binary	Toplama	`a + b`	İki değeri toplar
`-`	Binary	Çıkarma	`a - b`	İlk değerden ikinciyi çıkarır
`*`	Binary	Çarpma	`a * b`	İki değeri çarpar
`/`	Binary	Bölme	`a / b`	İlk değeri ikinciye böler
`%`	Binary	Modül (Kalan)	`a % b`	Tam sayı bölmesinden kalanı verir

1.2.2. Kod Üzerinde Gösterim

```c
#include <stdio.h>

int main() {
    int x = 10, y = 3;

    printf("Toplam: %d\n", x + y);        // 13
    printf("Fark:   %d\n", x - y);        // 7
    printf("Çarpım: %d\n", x * y);        // 30
    printf("Bölüm:  %d\n", x / y);        // 3 (Tamsayı bölmesi!)
    printf("Kalan:  %d\n", x % y);        // 1
    printf("Negatif x: %d\n", -x);        // -10

    return 0;
}
```

Önemli Noktalar:

1. **Tamsayı Bölmesi (Integer Division)**: Her iki işlenen de tamsayı ise, sonuç da tamsayı olur. Örneğin, `21 / 5` işleminin sonucu 5.25 değil, **5**'tir . Kesirli kısım atılır (truncation).

2. **Mod Operatörü (`%`)**: Sadece tamsayılarla kullanılabilir. `21 % 5` işlemi, kalan 1 değerini verir . Mekatronik uygulamalarda periyodik işlemlerde (örneğin bir motorun tur sayısını hesaplarken) sıklıkla kullanılır.

3. **Sıfıra Bölme Hatası**: Bir sayı sıfıra bölünmeye çalışılırsa, program çalışma zamanı hatası (runtime error) verir ve çöker. Bu durum her zaman kontrol edilmelidir.

1.3. İlişkisel (Karşılaştırma) Operatörler (Relational Operators)

İki değer arasındaki ilişkiyi test eden operatörlerdir. Sonuç olarak **1 (true/doğru)** veya **0 (false/yanlış)** döndürürler .

Operatör	Anlamı	Örnek	Sonuç (a=5, b=3 için)
`>`	Büyüktür	`a > b`	1 (doğru)
`<`	Küçüktür	`a < b`	0 (yanlış)
`>=`	Büyük veya eşit	`a >= b`	1 (doğru)
`<=`	Küçük veya eşit	`a <= b`	0 (yanlış)
`==`	Eşittir	`a == b`	0 (yanlış)
`!=`	Eşit değildir	`a != b`	1 (doğru)

Sık Yapılan Hatalar: `=` vs `==`

C programlamada sıklıkla karşılaşılan ve tespit edilmesi güç mantıksal hatalardan biridir.

1. Atama Operatörü (`=`)

Atama operatörü (`=`) bir değer döndürür. Bu değer, atanan değerin kendisidir.

```
1   if (x = 5) {
2       // Bu blok HER ZAMAN çalışır!
3       // Çünkü 'x = 5' ifadesinin sonucu 5'tir (sıfırdan farklı = TRUE).
4   }
```

Kod	Amaç	Sonuç
`if (x == 5)`	Eşitlik kontrolü	`x` 5 ise blok çalışır
`if (x = 5)`	Hata! Atama	`x` 'e 5 atanır ve blok sürekli çalışır

2. Noktalı Virgül Hatası

`if` ifadesinin sonuna yanlışlıkla konulan bir `;` programın mantığını tamamen bozar.

Hatalı Kullanım:

```
if (sicaklik > 100);
{
    printf("Sistem Aşırı Isındı!\n");
}
```

Ne Oluyor?

- Derleyici `if` satırını bitmiş kabul eder (boş bir komut çalıştırır).
- Altındaki parantezli blok `if` 'ten bağımsız bir kod bloğu olarak kabul edilir ve **her zaman** çalışır.
- Çıktı: Sıcaklık fark etmeksizin mesaj her zaman ekrana basılır.

3. Zincirleme Karşılaştırma

Matematikteki $5 < x < 10$ yazımı C dilinde beklediğiniz gibi çalışmaz.

Mantıksal Hatalı Yazım:

```
1  if (5 < x < 10) // Mantıksal hata!
```

C Nasıl Değerlendirir?

1. Önce `5 < x` değerlendirilir. Sonuç `0` (yanlış) veya `1` (doğru) olur.
2. Ardından bu `0` veya `1` değeri `10` ile karşılaştırılır: `(0 veya 1) < 10`.
3. Sonuç **her zaman 1 (True)** olur çünkü hem 0 hem 1, 10'dan küçüktür.

Doğru Yazım (Mantıksal Operatör İle):

```
1  if (x > 5 && x < 10)
```

1.4. Mantıksal Operatörler (Logical Operators)

Program akışında karar verme mekanizmalarının temelini oluşturan, birden fazla koşulun birleştirilmesiyle daha karmaşık mantıksal kontrollerin (örneğin "A koşulu sağlansın VE B koşulu da sağlansın") yapılmasını sağlayan operatörlerdir. İlişkisel operatörler gibi mantıksal operatörler de doğru (`1`) veya yanlış (`0`) değerleri üzerinde işlem yapar ve sonuç olarak yine mantıksal bir değer (`1` veya `0`) üretirler.

Operatör	Anlamı	Örnek	Açıklama
`&&`	VE (AND)	`(a > 5) && (b < 10)`	Her iki koşul da doğruysa doğru
`||`	VEYA (OR)	`(a > 5) || (b < 10)`	En az bir koşul doğruysa doğru
`!`	DEĞİL (NOT)	`!(a > 5)`	Koşulun tersi (a>5 yanlışsa doğru)

1.4.1. Kısa Devre Değerlendirmesi (Short-Circuit Evaluation)

Mantıksal operatörlerin çok önemli bir özelliği, sonucun kesinleştiği anda değerlendirmeyi durdurmalarıdır. `&&` (VE) operatöründe, ilk işlenen **yanlış (false)** ise, ikinci işlenen **hiç değerlendirilmez**. Çünkü sonuç kesinlikle yanlıştır.

1.4. Mantıksal Operatörler (Logical Operators) (Devamı)

- `||` (VEYA) operatöründe, ilk işlenen **doğru (true)** ise, ikinci işlenen **hiç değerlendirilmez**. Çünkü sonuç kesinlikle doğrudur.

Mühendislik Uygulaması - Sıfır Bölme Koruması:

```
int payda = 0;
int pay = 10;

if (payda != 0 && (pay / payda) > 2) {
    // Kısa devre sayesinde payda 0 olduğunda bölme işlemi HİÇ ÇALIŞTIRILMAZ
    // Program çökmez!
}
```

1.5. İleri Düzey Operatörler

1.5.1. Bit Düzeyi (Bitwise) Operatörler

C dili, genel yazılım geliştirmenin yanı sıra **donanım seviyesinde kontrol sağlayan** (Gömülü Sistemler) bir dildir.

- Normal `a + b` aritmetiği, sayıyı bir bütün olarak ele alır.
- **Bitwise (Bit Düzeyi) operatörler**, verilerin bellek (RAM) üzerindeki ikili (binary) gösterimlerine doğrudan erişerek her bir bit üzerinde bağımsız işlem yapılmasını sağlar.

Neden İhtiyacımız Var? Ne İşe Yarar?

1. **Donanım Kontrolü:** Sadece istenilen donanım pinindeki (örneğin sensör, motor) biti, etrafındaki bağımsız sistemleri bozmadan aktif/pasif etmemizi sağlar.

2. **Yüksek Performans:** Ağır çarpma ve bölme işlemlerini, bit kaydırma (shift) tekniği ile donanım seviyesinde çok daha düşük maliyetli ve hızlı gerçekleştirmemizi sağlar.

3. **Bellek Optimizasyonu:** Örneğin 8 adet farklı `True/False` (Açık/Kapalı) durum bilgisini 8 ayrı veri tipinde tutmak yerine, boyutları oldukça kısıtlı cihazlarda tek bir 8-bitlik (`char`) değişkenin bitlerine sıkıştırarak RAM'den tasarruf edilmesini sağlar.

4. **Ağ (Networking) ve Kriptografi:** Alt ağ (subnet) IP maskeleme işlemlerinde (AND) ve neredeyse tüm donanımsal şifreleme/veri doğrulama algoritmalarında (XOR) mantıksal temel taşıdır.

1.5.2. Bit Düzeyi Operatör Kataloğu

Sembol	İsim	İngilizce	Açıklama
`&`	Bit Düzeyi VE	Bitwise AND	Maskeleme, bit temizleme işlemi yapar
`|`	Bit Düzeyi VEYA	Bitwise OR	Bir biti aktif hale getirme (Set) eder
`^`	Bit Düzeyi XOR	Exclusive OR	Biti tersine çevirme (Toggle) yapar
`~`	TÜMLEYEN	Complement	"1"leri "0", "0"ları "1" yapar
`<<`	Sola Kaydırma	Left Shift	Bitleri sola kaydırır (Sayıyı *2^n yapar)
`>>`	Sağa Kaydırma	Right Shift	Bitleri sağa kaydırır (Sayıyı /2^n yapar)

1.5.2.1. Bitwise Operatörlerin Kullanımı (Basit Örnekler)

Değişkenlerin ikili (binary) sistemdeki karşılıkları alt alta yazılarak bağımsız bit işlemleri yapılır. Örnek olarak A = 5 (`0101`) ve B = 3 (`0011`) sayılarını ele alalım:

- VE (`&` - AND): Sadece her iki bit 1 olduğunda 1 üretir. *(Maskeleme)* `5 & 3` → `0101 & 0011 = 0001` (Sonuç: 1)
- VEYA (`|` - OR): Bitlerden en az biri 1 ise 1 üretir. *(Biti aktif etme)* `5 | 3` → `0101 | 0011 = 0111` (Sonuç: 7)
- ÖZEL VEYA (`^` - XOR): İki bit birbirinden farklıysa 1 üretir. *(Tersleme)* `5 ^ 3` → `0101 ^ 0011 = 0110` (Sonuç: 6)
- TÜMLEYEN (`~` - NOT): 1'leri 0, 0'ları 1 yapar. `~5` → `~00000101 = 11111010`

1.5.2.2. Mühendislikte Gerçek Dünya Uygulamaları (Gömülü Sistemler)

Bit işlemleri genellikle mikrodenetleyicilerde (Arduino, ARM vb.) doğrudan donanım pinlerini kontrol etmek için kullanılır. Örneğin 8 pini aynı anda kontrol edebildiğimiz bir `PORT` değişkeni (başlangıçta `0000 0000`) düşünelim:

- **Biti 1 Yapma (Set Bit)** $\longrightarrow$ `|` **(OR):** Sadece 3. pindeki cihazı / motoru çalıştırmak. `PORT = PORT | (1 << 3);` *(Diğer pinler etkilenmez, sadece 3. bit 1 olur)*
- **Biti 0 Yapma (Clear Bit)** $\longrightarrow$ `& ~` **(AND NOT):** Sadece 3. pindeki cihazı durdurmak. `PORT = PORT & ~(1 << 3);` *(Sadece 3. bit 0 olur, diğerleri aynı kalır)*
- **Durum Değiştirme (Toggle Bit)** $\longrightarrow$ `^` **(XOR):** LED yanıyorsa söndürmek, sönükse yakmak. `PORT = PORT ^ (1 << 3);` *(3. pinin durumunu tersine çevirir)*
- **Durum Kontrolü (Masking)** $\longrightarrow$ `&` **(AND):** 3. pindeki sensörden sinyal (1) geliyor mu? `if (PORT & (1 << 3)) { ... }` *(Diğer pinleri maskeler, sadece 3. bite bakar)*

1.5.3. Bit Kaydırma (Shift) İle Yüksek Performanslı İşlemler

Bitleri sola (`<<`) kaydırmak, donanımsal düzeyde sayıyı 2'nin kuvvetleriyle çarpmak anlamına gelir ve yüksek işlem hızı sağlar.

```
1  unsigned char yas = 5;  // Binary: 0000 0101 (Değer: 5)
2
3  unsigned char sonuc = yas << 2;
4  // Binary durumu sola doğru 2 birim hızla kaydırılır
5  // Yeni Binary: 0001 0100 (Değer: 20)
6  // Teorik sonuç = 5 * (2^2) = 20 !
```

1.5.3.1. Gerçek Hayat Örneği: Akıllı Ev Sistemi (Bitwise OR / AND)

Diyelim ki bir akıllı ev yapıyoruz ve elimizde 8 odanın ışık durumunu tutan tek bir değişken var (`Mutfak = 1. bit` , `Salon = 2. bit` , `Yatak Odası = 3. bit` vs.). Tüm bu odaları tek bir sayıda tutarak harika bir bellek optimizasyonu sağlayabiliriz.

```c
// Başlangıçta tüm ışıklar kapalı (0000 0000)
unsigned char ev_isiklari = 0;

// SADECE Salon'un (2. bit) ışığını açmak (Bitwise OR '|')
// Diğer hiçbir odanın ışığı etkilenmez!
ev_isiklari = ev_isiklari | (1 << 1);

// Salon'un ışığı açık mı diye kontrol etmek (Bitwise AND '&')
if (ev_isiklari & (1 << 1)) {
    printf("Salon isigi su an acik!\n");
}
```

1.5.4. sizeof() Operatörü

`sizeof` kelimesi bir fonksiyon gibi görünse de, tıpkı `+` veya `-` gibi gömülü bir **operatördür**.

İşlevi: Program çalışma zamanında bir değişkenin veya veri tipinin ilgili donanım mimarisinde bellekte (RAM) **kaç byte yer kapladığını** (bellek boyutunu) döndürür.

```
1  short s_deger = 10;
2  printf("short belleği: %lu", sizeof(short));   // Çıktı: 2 Bytes
3  printf("değişkenin belleği: %lu", sizeof(s_deger)); // Çıktı: 2 Bytes
```

1.5.5. Virgül (,) Operatörü

Virgül operatörü (`,`), C dilindeki operatörler arasında öncelik sıralamasının en altında yer alır. İfadeleri tek satırda ardışık yazıp yan yana icra etmek için kullanılır.

```
1  int i=0, j=5;
2
3  // Döngü yapılarındaki gizli kullanım ve Sağın Atanması
4  int sonuc = (i=3, j=5+i, i+j);
```

Önce i=3 atanır. Sonra j=8 atanır. En son i+j (3+8=11) olur. Virgülün çalışma esası gereği en sağdaki 11 değeri `sonuc`'a atanır.

1.6. Arttırma ve Azaltma Operatörleri (Increment/Decrement Operators)

C dilinin en güçlü ama aynı zamanda en dikkatli kullanılması gereken operatörlerindendir .

Operatör	İsim	Örnek	Açıklama
`++`	Ön-arttırma (pre-increment)	`++x`	x önce arttırılır, sonra değeri kullanılır
`++`	Sonra-arttırma (post-increment)	`x++`	x'in değeri önce kullanılır, sonra arttırılır
`--`	Ön-azaltma (pre-decrement)	`--x`	x önce azaltılır, sonra değeri kullanılır
`--`	Sonra-azaltma (post-decrement)	`x--`	x'in değeri önce kullanılır, sonra azaltılır

Kritik Fark:

```
int x = 5;
int y = ++x;  // x = 6, y = 6 (x arttırılır, SONRA y'ye atanır)
int a = 5;
int b = a++;  // b = 5, a = 6 (önce a'nın değeri b'ye atanır, SONRA a arttırılır)
```

Uyarı - Yan Etki (Side Effect):

Arttırma/azaltma operatörleri, değişkenin değerini değiştirir (yan etki oluşturur). Aynı ifade içinde bir değişken birden fazla kez değiştirilmemelidir .

```
int i = 5;
int j = i++ + ++i;  // TANIMSIZ DAVRANIŞ (undefined behavior)!
// Farklı derleyicilerde farklı sonuçlar alınabilir.
```

1.7. Atama Operatörleri (Assignment Operators)

Temel atama operatörü `=` yanında, işlem ve atamayı birleştiren bileşik atama operatörleri de vardır .

Operatör	Örnek	Anlamı
`=`	`x = 5`	x'e 5 ata
`+=`	`x += 3`	`x = x + 3`
`-=`	`x -= 4`	`x = x - 4`
`*=`	`x *= 2`	`x = x * 2`
`/=`	`x /= 3`	`x = x / 3`
`%=`	`x %= 5`	`x = x % 5`

Avantajı: Bileşik atama operatörleri, özellikle işlenen karmaşık bir ifadeyse (örneğin dizi indeksi veya pointer) daha verimli kod üretilmesini sağlar ve okunabilirliği arttırır .

1.8. Operatör Önceliği (Precedence) ve Birleşme Özelliği (Associativity)

Bir ifadede birden fazla operatör olduğunda, hangi işlemin önce yapılacağını **öncelik (precedence)** kuralları belirler .

Öncelik	Operatörler	Açıklama	Birleşme
1 (En Yüksek)	`()` `[]` `->` `.`	Fonksiyon çağrısı, dizi indeksi, üye erişimi	Soldan sağa
2	`!` `~` `++` `--` `+` `-` `*` `&` `sizeof`	Unary operatörler	Sağdan sola
3	`*` `/` `%`	Çarpma, bölme, mod	Soldan sağa
4	`+` `-`	Toplama, çıkarma	Soldan sağa
5	`<<` `>>`	Bit kaydırma	Soldan sağa

Öncelik	Operatörler	Açıklama	Birleşme
6	`<` `<=` `>` `>=`	İlişkisel (karşılaştırma)	Soldan sağa
7	`==` `!=`	Eşitlik kontrolü	Soldan sağa
8	`&`	Bitwise AND	Soldan sağa
9	`^`	Bitwise XOR	Soldan sağa
10	`|`	Bitwise OR	Soldan sağa

Öncelik	Operatörler	Açıklama	Birleşme
11	`&&`	Mantıksal VE	Soldan sağa
12	`||`	Mantıksal VEYA	Soldan sağa
13	`? :`	Koşul operatörü	Sağdan sola
14	`=` `+=` `-=` vb.	Atama operatörleri	Sağdan sola
15 (En Düşük)	`,`	Virgül operatörü	Soldan sağa

Altın Kural: Karmaşık ifadelerde, öncelik tablosuna güvenmek yerine **parantez kullanın**. Bu, kodunuzu daha okunabilir ve hatasız yapar .

Örnek - Öncelik Hatası:

```
int mask = 0x01;
if (mask & 0x10 == 16) {  // YANLIŞ! == operatörü, &'den önce çalışır
    // Bu blok asla çalışmaz
}

if ((mask & 0x10) == 16) { // DOĞRU! Önce & işlemi, sonra karşılaştırma
    // Şimdi doğru çalışır
}
```

1.9. Tip Dönüşümleri (Type Conversions)

1.8.1. Otomatik (Implicit) Tip Dönüşümü Farklı tiplerin bir arada kullanıldığı ifadelerde, derleyici belirli kurallara göre otomatik dönüşüm yapar :

```
int a = 5;
float b = 2.5;
float sonuc = a + b;  // a otomatik olarak float'a dönüştürülür
```

1.8.2. Tamsayı Yükseltmesi (Integer Promotion) `char` ve `short` tipleri, işlemlerde otomatik olarak `int`'e yükseltilir.

```
char a = 100;
char b = 99;
char c = a * b; // HATA! a ve b int'e yükseltilir, çarpım int olur.
                // Sonuç char'a sığmaz, taşma olur.
```

1.8.3. Bilinçli (Explicit) Tip Dönüşümü - Casting Programcının manuel olarak tip dönüşümü yapmasıdır .

```
int a = 5, b = 2;
float sonuc1 = a / b;        // sonuc1 = 2.0 (önce tamsayı bölmesi, sonra atama)
float sonuc2 = (float)a / b; // sonuc2 = 2.5 (önce a float'a çevrilir, sonra bölme)
```

Mühendislik Uygulaması - Taşma (Overflow) Riski:

```
int a = 1000000;
int b = 1000000;
int c = (a * b) / 1000000;  // RİSKLİ! a*b işlemi int sınırını aşabilir (taşma)
// 32-bit sistemde sonuç -727 gibi anlamsız bir değer olabilir

long long d = (long long)a * b / 1000000; // DOĞRU: Önce long long'a çevir, sonra çarp
```

1.10. İfade (Expression) Kavramı

İfade, değişkenler, sabitler ve operatörlerin bir araya gelerek yeni bir değer ürettiği yapıdır .

İfade Türleri:

1. Aritmetik ifadeler: `a + b * c`
2. İlişkisel ifadeler: `x > y`
3. Mantıksal ifadeler: `(a > b) && (c < d)`
4. Atama ifadeleri: `x = 5` (değeri 5'tir)
5. Fonksiyon çağrıları: `printf("Merhaba")` (dönüş değeri vardır)

BÖLÜM 2: OPERATÖRLER UYGULAMA

2.1. Uygulama 1: Operatör Önceliği ve Birleşme Özelliği Testi

Amaç: Operatör önceliği ve birleşme özelliğinin pratikte nasıl çalıştığını gözlemlemek.

```c
#include <stdio.h>

int main() {
    int a = 5, b = 3, c = 2;
    int sonuc;

    printf("=== OPERATOR ONCELIGI TESTI ===\n");

    // Test 1: Çarpma önce, toplama sonra
    sonuc = a + b * c;  // 5 + (3*2) = 5 + 6 = 11
    printf("%d + %d * %d = %d\n", a, b, c, sonuc);

    // Test 2: Parantez önceliği değiştirir
    sonuc = (a + b) * c;  // (5+3)*2 = 8*2 = 16
    printf("(%d + %d) * %d = %d\n", a, b, c, sonuc);

    // Test 3: Soldan sağa birleşme (ayrı işlemler)
    sonuc = a - b - c;  // ((5-3)-2) = 2-2 = 0
```

Kod Devamı...

```c
    printf("%d - %d - %d = %d\n", a, b, c, sonuc);

    // Test 4: Karışık mantıksal ve ilişkisel operatörler
    int x = 10, y = 5, z = 8;
    int mantiksal = (x > y) && (y < z) || (x == z);
    // Önce: (x>y) doğru(1), (y<z) doğru(1) → 1 && 1 = 1
    // Sonra: (x==z) yanlış(0) → 1 || 0 = 1
    printf("(x>y) && (y<z) || (x==z) = %d\n", mantiksal);

    return 0;
}
```

2.2. Uygulama 2: Arttırma/Azaltma Operatörleri ve Yan Etkiler

Amaç: Pre-increment ve post-increment arasındaki farkı anlamak.

```c
#include <stdio.h>

int main() {
    printf("=== PRE-INCREMENT vs POST-INCREMENT ===\n\n");

    int i = 5, j = 5;
    int sonuc1, sonuc2;

    printf("Başlangıç: i = %d, j = %d\n", i, j);

    // Post-increment
    sonuc1 = i++;
    printf("sonuc1 = i++ -> sonuc1 = %d, i = %d (i sonra arttı)\n", sonuc1, i);

    // Pre-increment
    sonuc2 = ++j;
    printf("sonuc2 = ++j -> sonuc2 = %d, j = %d (j önce arttı)\n\n", sonuc2, j);
```

Kod Devamı...

```c
    // Döngü içinde kullanım
    printf("Döngü örnekleri:\n");
    printf("Post-increment ile:\n");
    i = 1;
    while (i <= 3) {
        printf("i = %d (kullanım anı), ", i++);
        printf("i'nin yeni değeri: %d\n", i);
    }

    printf("\nPre-increment ile:\n");
    j = 1;
    while (j <= 3) {
        printf("j = %d (kullanım anı), ", ++j);
        printf("j'nin yeni değeri: %d\n", j);
    }

    // UYARI: Aynı ifadede birden fazla yan etki
    printf("\n=== UYARI: TANIMSIZ DAVRANIŞ ===\n");
```

Kod Devamı...

```
1       int k = 5;
2       // int belirsiz = k++ + ++k;  // YORUM SATIRINDAN ÇIKARMAYIN! (undefined behavior)
3       printf("k++ + ++k gibi ifadeler tanımsızdır. Derleyiciye göre değişir!\n");
4
5       return 0;
6   }
```

2.3. Uygulama 3: Tip Dönüşümleri ve Tamsayı Bölmesi

Amaç: Otomatik ve bilinçli tip dönüşümlerini anlamak, tamsayı bölmesinin inceliklerini kavramak.

```c
#include <stdio.h>

int main() {
    printf("=== TAMSAYI BOLMESI ve TIP DONUSUMLERI ===\n\n");

    int a = 21, b = 5;
    float sonuc;

    // Tamsayı bölmesi
    sonuc = a / b;
    printf("1. int / int = int: %d / %d = %f (beklenen 4.2 değil!)\n", a, b, sonuc);
    printf("   Açıklama: Önce tamsayı bölmesi (21/5=4), sonra float'a atama\n\n");

    // Cast ile düzeltme
    sonuc = (float)a / b;
    printf("2. (float)/int = float: (float)%d / %d = %f\n", a, b, sonuc);

    sonuc = a / (float)b;
```

Kod Devamı...

```c
    printf("3. int/(float) = float: %d / (float)%d = %f\n\n", a, b, sonuc);

    // Mod operatörü
    int kalan = a % b;
    printf("Mod operatörü: %d %% %d = %d (kalan)\n\n", a, b, kalan);

    // Karışık tip ifadeleri
    int x = 5;
    double y = 2.0;
    double z = x + y;  // x otomatik double'a çevrilir
    printf("int + double = double: %d + %.1f = %.1f\n\n", x, y, z);

    // Büyük sayılarla taşma örneği
    int buyuk1 = 1000000;
    int buyuk2 = 1000000;
    int tasma = buyuk1 * buyuk2 / 1000000;
    printf("TAŞMA ÖRNEĞİ:\n");
    printf("%d * %d / 1000000 = %d (beklenen 1000000)\n", buyuk1, buyuk2, tasma);
```

Kod Devamı...

```
1
2       long long dogru = (long long)buyuk1 * buyuk2 / 1000000;
3       printf("Cast ile düzeltme: (long long)%d * %d / 1000000 = %lld\n\n", buyuk1, buyuk2, d
4
5       return 0;
6   }
```

2.4. Uygulama 4: Kısa Devre Değerlendirmesi

Amaç: Mantıksal operatörlerde kısa devre özelliğini anlamak.

```c
#include <stdio.h>

int main() {
    printf("=== KISA DEVRE DEGERLENDIRMESI ===\n\n");

    int a = 0, b = 5, c = 10;
    int sonuc;

    // && operatörü - ilk operand false ise ikinci değerlendirilmez
    printf("&& KISA DEVRE:\n");
    printf("Önce: a = %d, b = %d, c = %d\n", a, b, c);

    sonuc = (a != 0) && (b++ > 3);
    // a != 0 yanlış (false) olduğu için b++ ASLA çalışmaz!

    printf("sonuc = (a != 0) && (b++ > 3) -> sonuc = %d\n", sonuc);
    printf("Sonra: a = %d, b = %d (b artmadı!)\n\n", a, b);
```

Kod Devamı...

```c
    // Sıfır bölme koruması (mühendislik uygulaması)
    int payda = 0;
    int pay = 10;

    printf("SIFIR BOLME KORUMASI:\n");
    printf("payda = %d, pay = %d\n", payda, pay);

    if (payda != 0 && (pay / payda) > 2) {
        printf("Bu mesaj asla yazdırılmaz\n");
    } else {
        printf("Güvenli kontrol: payda != 0 olduğu için bölme yapılmadı, program çökmedi!\
    }

    // || operatörü - ilk operand true ise ikinci değerlendirilmez
    int x = 1, y = 5;
    printf("\n|| KISA DEVRE:\n");
    printf("Önce: x = %d, y = %d\n", x, y);
```

Kod Devamı...

```
1       sonuc = (x == 1) || (y++ > 10);
2       // x == 1 doğru (true) olduğu için y++ ASLA çalışmaz!
3
4       printf("sonuc = (x == 1) || (y++ > 10) -> sonuc = %d\n", sonuc);
5       printf("Sonra: x = %d, y = %d (y artmadı!)\n", x, y);
6
7       return 0;
8   }
```

2.5. Uygulama 5: Bütünleşik Mühendislik Uygulaması - PID Katsayı Hesaplama

Amaç: Öğrenilen tüm operatörleri kullanarak bir mühendislik problemini çözmek.

```c
#include <stdio.h>

int main() {
    // PID kontrolör katsayı hesaplama simülasyonu
    double Kp = 2.5, Ki = 0.1, Kd = 0.05;
    double oran = 0.0;
    int secim;
    double hata = 10.5, onceki_hata = 8.2, integral = 15.3;
    double P, I, D, PID_cikis;

    printf("========================================\n");
    printf("   PID KONTROLOR KATSAYI HESAPLAMA     \n");
    printf("========================================\n\n");

    printf("Mevcut PID katsayıları:\n");
    printf("Kp = %.2f, Ki = %.2f, Kd = %.2f\n\n", Kp, Ki, Kd);

    printf("Mevcut hata değerleri:\n");
```

Kod Devamı...

```c
    printf("Hata = %.2f, Önceki Hata = %.2f, İntegral = %.2f\n\n", hata, onceki_hata, inte

    // Kullanıcıya seçenekler sun
    printf("İşlem seçin:\n");
    printf("1 - Oransal (P) hesapla\n");
    printf("2 - İntegral (I) hesapla\n");
    printf("3 - Türev (D) hesapla\n");
    printf("4 - Tüm PID çıkışını hesapla\n");
    printf("Seçiminiz (1-4): ");
    scanf("%d", &secim);

    // İşlem seçimine göre hesaplama
    if (secim == 1) {
        P = Kp * hata;
        printf("\nOransal (P) çıkışı: Kp * hata = %.2f * %.2f = %.2f\n", Kp, hata, P);
    }
    else if (secim == 2) {
        I = Ki * integral;
```

Kod Devamı...

```
        printf("\nİntegral (I) çıkışı: Ki * integral = %.2f * %.2f = %.2f\n", Ki, integral
    }
    else if (secim == 3) {
        double hata_farki = hata - onceki_hata;
        D = Kd * hata_farki;
        printf("\nTürev (D) çıkışı: Kd * (hata - önceki_hata) = %.2f * (%.2f - %.2f) = %.2
               Kd, hata, onceki_hata, D);
    }
    else if (secim == 4) {
        P = Kp * hata;
        I = Ki * integral;
        double hata_farki = hata - onceki_hata;
        D = Kd * hata_farki;
        PID_cikis = P + I + D;

        printf("\n=== PID ÇIKIŞ HESABI ===\n");
        printf("P = Kp * hata          = %.2f * %.2f = %.2f\n", Kp, hata, P);
        printf("I = Ki * integral      = %.2f * %.2f = %.2f\n", Ki, integral, I);
```

Kod Devamı...

```
        printf("D = Kd * (hata - önceki) = %.2f * (%.2f - %.2f) = %.2f\n", Kd, hata, oncek
        printf("-----------------------------------------------\n");
        printf("PID Çıkış = P + I + D  = %.2f + %.2f + %.2f = %.2f\n", P, I, D, PID_cikis)

        // Mod operatörü kullanarak periyodik kontrol
        int iterasyon = 0;
        printf("\nPID çıkışının periyodik kontrolü:\n");
        for (iterasyon = 1; iterasyon <= 10; iterasyon++) {
            if (iterasyon % 3 == 0) {  // Her 3 iterasyonda bir
                printf("Iterasyon %d: PID çıkışı = %.2f (KONTROL ZAMANI)\n", iterasyon, PI
            } else {
                printf("Iterasyon %d: PID çıkışı = %.2f\n", iterasyon, PID_cikis);
            }
        }
    }
    else {
        printf("Geçersiz seçim!\n");
    }
```

Kod Devamı...

```
1
2       // Operatör önceliği gösterimi
3       printf("\n=== OPERATOR ONCELIGI NOTU ===\n");
4       printf("PID hesabında: P + I + D = P + (I + D) değil, (P + I) + D (soldan sağa)\n");
5       printf("Ancak toplama işlemi birleşmeli olduğu için sonuç aynıdır.\n");
6
7       return 0;
8   }
```

Soru Cevap

Bilgisayar Programlama